



crusts v0.0.2

CRustS - From Unsafe C to safer Rust

[#refactoring](#) [#c2rust](#) [#rust](#)

Follow

[Readme](#)

[2 Versions](#)

[Dependencies](#)

[Dependents](#)

CRustS - Transpiling Unsafe C code to Safer Rust

Michael Ling, Yijun Yu, Haitao Wu, Yuan Wang, James R. Cordy, Ahmed E. Hassan
Trustworthiness Software Engineering & Open Source Lab
Huawei Technologies, Inc.

As a safer alternative to C, Rust is a language for programming system software with a type-safe compiler to check its memory and concurrency safety. To facilitate a smooth transition from C to Rust in an existing project, and lay a solid foundation for an initial Rust re-implementation of existing functionalities in C, it would be helpful to have a source-to-source transpiler that can transform programs from C to Rust using program transformation technologies. However, existing C-to-Rust transformation tool sets have the drawback that they largely preserve the unsafe semantics of C, while rewriting them in Rust syntax. As such, the safety of the resulting Rust programs still depends primarily on the programmers, rather than on the Rust compiler. To gain more safety guarantees, in this demo, we present CRustS a systematic source-to-source transformation approach to increase the ratio of the code passing the safety checks of Rust compiler by relaxing the semantics-preserving constraints of the transformation. Our method uses 220 [TXL](#) source-to-source transformation rules, of which 198 are strictly semantics-preserving and 22 are semantics approximating, thus reducing the scope of unsafe expressions and exposing more opportunities for safe refactoring. Our method has been evaluated on both open-source and commercial projects. Our solution demonstrates significantly higher safe code ratios after the transformations, with function-level safe code ratios comparable to the average level of idiomatic Rust projects.

Compared to the [Laertes](#)[OOPSLA'21], with respect to their own benchmarks, the safe ratio obtained by CRustS is much higher.

Installation

```
# install c2rust
if [ $(uname -s) == "Darwin" ]; then
  git clone https://github.com/immunant/c2rust
  cd c2rust
  scripts/provision_mac.sh
  cargo build --release
  cp target/release/c2rust $HOME/.cargo/bin
  cp target/release/c2rust-transpile $HOME/.cargo/bin
  cp target/release/c2rust-analyze $HOME/.cargo/bin
  cp target/release/c2rust-instrument $HOME/.cargo/bin
  cd -
  brew install bear
  export PATH=$HOME/.local/bin:$HOME/.cargo/bin:$PATH

  cargo install crusts
```

```

elif [ $(uname -s) == "Linux" ]; then
    apt install llvm cmake clang libclang-dev bear -y
    LLVM_LIB_DIR=/usr/lib/llvm-14/lib/ cargo install c2rust
    curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
    rustup override set nightly-2021-11-22-x86_64-unknown-linux-gnu
    rustup component add rustfmt --toolchain nightly-2021-11-22-x86_64-unknown-linux-gnu
    export PATH=$HOME/.local/bin:$HOME/.cargo/bin:$PATH

    cargo install crusts

else

    docker pull yijun/crusts

fi

```

Usage:

Run `crusts` in the folder where there is a `Makefile`, using

```
crusts [-v | -c2rust]
```

or

```
docker run -v $(pwd):/mnt -t yijun/crusts [-v | -c2rust]
```

As a result, Rust code will be generated from the C code:

```

src/*.rs -- contains transpiled and refactored Rust code from the C code;
Cargo.toml build.rs lib.rs -- contains the `cargo build` configurations;

```

Options

- `-v` -- version information
- `-c2rust` -- only run `c2rust`

References

- Michael Ling, Yijun Yu, Haitao Wu, Yuan Wang, James Cordy, Ahmed Hassan. "In Rust We Trust: A transpiler from [Unsafe C to Safer Rust](#)", In: Proceedings of ICSE, 2022.
- Mehmet Emre, Ryan Schroeder, Kyle Dewey, and Ben Hardekopf. 2021. [Translating C to safer Rust](#). Proc. ACM Program. Lang. 5, OOPSLA, Article 121 (October 2021), 29 pages. ([code](#))
- [TXL](#)
- [C2Rust](#)

Update

- ☒ [comparing to Laertes](#)
- ☒ integrating with docker for Windows users
- ☒ adding a switch `-c2rust` to turn off refactoring
- ☒ adding a switch `-v` to show versioning
- ☐ bugfix: deref pointers, see `test_unsafe`
- ☐ bugfix: printf patterns, see `test_stdio`

📅 about 2 years ago

📅 2021 edition

📄 Apache-2.0

📦 18.7 KiB

Install

```
cargo install crusts
```

Running the above command will globally install the **crusts** binary.

Documentation

📖 docs.rs/crusts

Repository

📄 github.com/yijunyu/crusts

Owners



Yijun Yu

Report crate

Stats Overview

↓ **2,035**

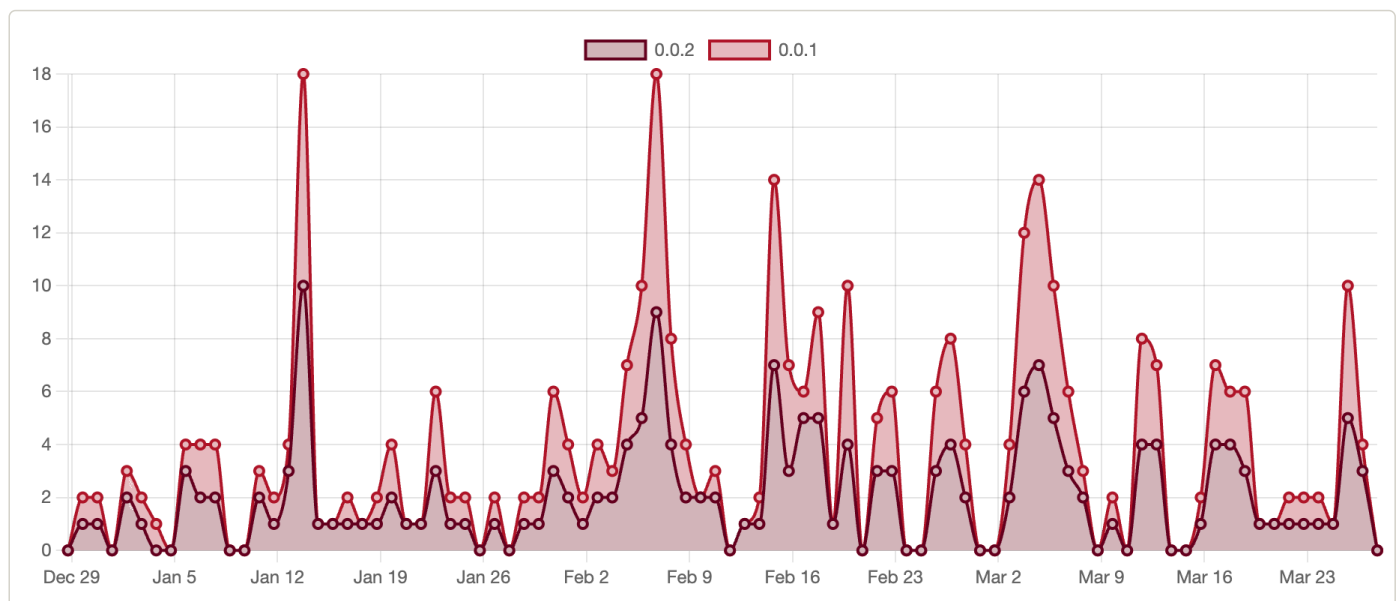
Downloads all time

📦 **2**

Versions published

Downloads over the last 90 days

Display as Stacked ▾



Rust

rust-lang.org

Rust Foundation

The crates.io team

Get Help

[The Cargo Book](#)

[Support](#)

[System Status](#)

[Report a bug](#)

Policies

[Usage Policy](#)

[Security](#)

[Privacy Policy](#)

[Code of Conduct](#)

[Data Access](#)

Social

[rust-lang/crates.io](#)

[#t-crates-io](#)

[@cratesiostatus](#)